BIT 102 – C Programming
Unit 1: Introduction to C Programming
Semester: I | **Credit Hours:** 3 | **Unit Duration:** 3 Hours | **Nature:** Theory + Laboratory
Unit 1 Syllabus

1. History and advantages of C
2. Problem analysis
3. Algorithm
4. Flowchart
5. Structure of a C program
6. Writing, compiling, debugging, executing and testing a C program
7. C programming environment in Windows and Unix/Unix-like systems

---

1. Introduction to Programming
A **computer program** is a set of instructions given to a computer to perform a particular task.
For example, if we want a computer to calculate the total marks of a student, we need to provide instructions such as:
Input marks → Calculate total → Display total
Example
Suppose a BIT student obtains:

- Programming = 75
- Mathematics = 68
- Computer Network = 72

The computer can calculate:
Total = 75 + 68 + 72 = 215
A C program can perform this calculation automatically.
Programming Process
A simple programming process can be represented as:
Problem
↓
Problem Analysis
↓
Algorithm
↓
Flowchart / Program Design
↓
Coding in C
↓
Compilation
↓
Debugging
↓
Execution
↓
Testing
↓
Correct Output

---

2. What is C?

C is a general-purpose, structured, procedural programming language used for developing a wide range of software applications.

C was designed to provide a relatively simple language that gives programmers considerable control over computer memory and hardware.

Examples of applications where C is important

- Operating systems
- Embedded systems
- System software
- Device drivers
- Compilers
- Networking software
- Database systems
- IoT devices
- High-performance applications

---

3. History of C

C has a significant role in the history of modern programming languages.

Development history

BCPL
↓
B
↓
C
↓
C++
↓
Many modern languages influenced by C

Dennis Ritchie

Dennis Ritchie developed C at **Bell Laboratories** in the early 1970s.

C was closely associated with the development of the **UNIX operating system**.

The language was designed to combine:

- High-level programming features
- Efficient execution
- Low-level memory manipulation
- Portability

Important historical milestones

Period	Development
1960s	Languages such as BCPL influenced later system programming
Around 1970	B language developed at Bell Labs
Early 1970s	C developed by Dennis Ritchie
1970s	UNIX increasingly developed using C
1978	*The C Programming Language* by Kernighan and Ritchie helped popularize C
1989	ANSI C standard, commonly known as C89
1990	ISO standardized C as C90
1999	C99 standard
2011	C11 standard
2018	C17 standard
2024	C23 standard

Exam point: Dennis Ritchie is widely recognized as the creator/developer of the C programming language.

---

4. Why is C Still Important?

Although many newer programming languages exist, C remains important because it provides a strong foundation for understanding how computers and programs work.

For a first-semester BIT student, learning C helps develop concepts such as:

- Variables
- Data types
- Operators
- Control structures
- Functions
- Arrays
- Pointers
- Memory management
- Structured programming

These concepts are useful when learning languages such as C++, Java, C#, and other programming technologies.

---

5. Advantages of C

5.1 Simple and Relatively Small Language

C has a relatively small set of core language features.

For example:

```
#include <stdio.h>

int main()
{
   printf("Hello Nepal!");
   return 0;
}
```

The basic structure is easy to understand once the syntax is learned.

5.2 Structured Programming

C supports structured programming.

A large problem can be divided into smaller logical parts using:

- Functions
- Selection statements
- Loops
- Blocks

For example:

```
Student Result System
      |
      +---- Input Marks
      |
      +---- Calculate Total
      |
      +---- Calculate Percentage
      |
      +---- Determine Grade
      |
      +---- Display Result
```

This makes a program easier to develop and maintain.

5.3 Portable

A C program can often be compiled on different operating systems with relatively few changes, provided a suitable C compiler is available.

For example, a C program may be compiled on:

- Windows
- Linux
- macOS
- Unix-like systems

5.4 Fast and Efficient

C programs can execute efficiently because C provides relatively direct control over computer resources.

This makes C useful for:

- Operating systems
- Embedded systems
- System software
- Performance-sensitive applications

5.5 Rich Operators

C provides many operators, including:

Arithmetic + - * / %
Relational > < >= <= == !=
Logical && || !
Assignment = += -= *= /=
Increment ++
Decrement --
Bitwise & | ^ << >>

These operators will be studied in detail in later units.

5.6 Extensible

C allows programmers to create functions and organize programs into reusable components.

For example:

```
int square(int n)
{
   return n * n;
}
```

The function can be called whenever a square needs to be calculated.

5.7 Middle-Level Language

C is sometimes described as a **middle-level language** because it combines features associated with high-level programming with relatively low-level capabilities.

High-level features
↑
|
C
|
↓
Low-level / hardware-oriented features

This does **not** mean C is formally classified as a separate "middle-level" category by programming-language standards; it is a common educational description.

---

6. Disadvantages of C
C also has some limitations.
6.1 No built-in object-oriented programming
C is primarily a procedural language. Features such as classes and inheritance are associated with C++ and other object-oriented languages.
6.2 Manual memory management
Programmers must manage dynamically allocated memory carefully.
6.3 Less protection from programming errors
Incorrect use of pointers or memory can cause serious problems.
6.4 No automatic garbage collection
Unlike some languages, C does not automatically reclaim dynamically allocated memory.
6.5 String handling is less convenient
C uses character arrays and library functions for many string operations.

---

7. Problem Analysis
Before writing a program, we should first understand the problem.
Problem analysis means identifying:

1. What is given?
2. What is required?
3. What processing is needed?
4. What output should be produced?

A useful model is:
INPUT → PROCESS → OUTPUT

Example 1: Calculate Area of a Rectangle
Problem
Write a program to calculate the area of a rectangle.
Step 1: Identify input
We need:

- Length
- Breadth

Step 2: Identify process
Formula:
Area = Length × Breadth
Step 3: Identify output
Display:
Area of rectangle
Therefore:
INPUT
Length, Breadth
↓
PROCESS
Area = Length × Breadth
↓
OUTPUT
Area

---

8. IPO Model
IPO = Input – Process – Output
It is a simple technique for analyzing programming problems.

Example: Student Total Marks

Component	Description
Input	Marks in three subjects
Process	Add the three marks
Output	Total marks

Input
↓
English, Mathematics, C Programming
↓
Process
Total = English + Mathematics + C Programming
↓
Output
Total Marks

---

9. Algorithm

An **algorithm** is a finite sequence of clear and unambiguous steps used to solve a problem.
In simple words:
An algorithm is a step-by-step procedure for solving a problem.

Characteristics of an Algorithm

A good algorithm generally has the following characteristics:

1. Input
It may accept zero or more inputs.
2. Output
It should produce at least one result or output for a useful computational problem.
3. Definiteness
Every step should be clear and unambiguous.
4. Finiteness
The algorithm should terminate after a finite number of steps.
5. Effectiveness
Each step should be practical and executable.

---

10. Example of Algorithm

Problem
Calculate the area of a rectangle.
Algorithm
Step 1: Start
Step 2: Read length
Step 3: Read breadth
Step 4: Calculate area = length × breadth
Step 5: Display area
Step 6: Stop

---

11. Algorithm for Student Average

Problem
Calculate the average marks of three subjects.
Algorithm
Step 1: Start
Step 2: Read marks of Subject 1

Step 3: Read marks of Subject 2
Step 4: Read marks of Subject 3
Step 5: Calculate total = m1 + m2 + m3
Step 6: Calculate average = total / 3
Step 7: Display total and average
Step 8: Stop

---

12. Advantages of Algorithms

Algorithms help programmers to:

- Understand the problem before coding
- Identify required inputs and outputs
- Detect logical errors early
- Communicate the solution to others
- Convert the solution into any programming language
- Simplify complex problems

Important point

An algorithm is **language-independent**.
For example, the same algorithm can be implemented using:

- C
- C++
- Java
- Python

---

13. Flowchart

A **flowchart** is a graphical representation of an algorithm or problem-solving process.
It uses standardized symbols to show:

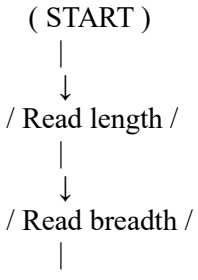
- Start and end
- Input/output
- Processing
- Decisions
- Flow of control

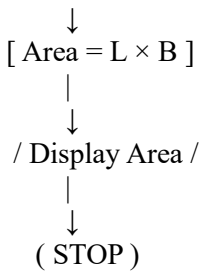
---

14. Common Flowchart Symbols

Symbol	Name	Purpose
Oval	Terminator	Start/Stop
Rectangle	Process	Calculation or processing
Parallelogram	Input/Output	Reading or displaying data
Diamond	Decision	Yes/No or True/False condition
Arrow	Flow line	Shows direction
Circle	Connector	Connects parts of a flowchart

Simplified representation





---

15. Flowchart Example: Check Even or Odd

Problem

Determine whether a number is even or odd.

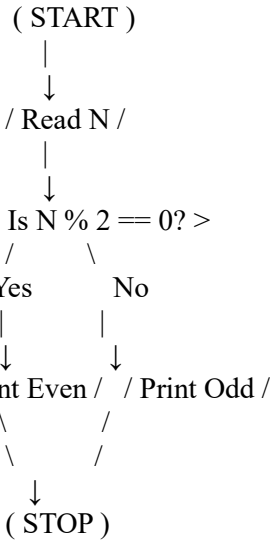
Logic

A number is even if:

Number % 2 == 0

Otherwise, it is odd.

Flowchart



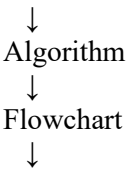
The **diamond** represents a decision.

---

16. Algorithm vs Flowchart

Algorithm	Flowchart
Written step-by-step	Graphical representation
Uses natural language/pseudocode	Uses symbols
Easy to write	Easy to visualize
Good for detailed logical steps	Good for understanding program flow
Language-independent	Language-independent

Simple relationship



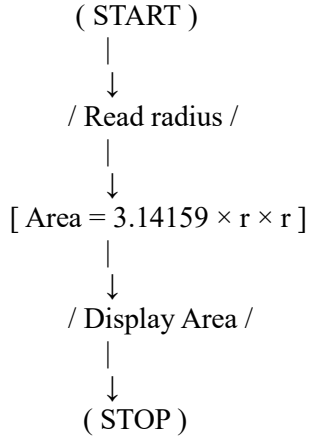
C Program

---

17. From Problem to C Program
Let's understand the complete process using a simple example.
Problem
Calculate the area of a circle.
Step 1: Analysis
Input: Radius r
Process:
Area = π × r × r
Output: Area

Step 2: Algorithm
Step 1: Start
Step 2: Read radius
Step 3: Calculate area = 3.14159 × radius × radius
Step 4: Display area
Step 5: Stop

Step 3: Flowchart



Step 4: C Program

```
#include <stdio.h>

int main()
{
   float radius, area;

   printf("Enter radius: ");
   scanf("%f", &radius);

   area = 3.14159 * radius * radius;

   printf("Area = %.2f", area);

   return 0;
}
```

Sample Output
Enter radius: 5
Area = 78.54
This demonstrates the complete programming process:
Problem
↓
Analysis
↓
Algorithm
↓
Flowchart
↓
Coding
↓
Compilation
↓
Execution
↓
Testing

---

18. Structure of a C Program
Consider the following program:

```
#include <stdio.h>

int main()
{
   int a, b, sum;

   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);

   sum = a + b;

   printf("Sum = %d", sum);

   return 0;
}
```

Let's understand its structure.

18.1 Preprocessor Directive

#include <stdio.h>

#include tells the preprocessor to include the contents/interface of the specified header file.

stdio.h provides declarations for standard input/output functions such as:

printf()

scanf()

18.2 Main Function

int main()

main() is the function where program execution begins in a hosted C program.

```
{
   ...
}
```

The braces define the body of the function.

18.3 Variable Declaration

int a, b, sum;

Three integer variables are declared:

- a
- b
- sum

Detailed discussion of data types and variables comes in **Unit 2**.

18.4 Input Statement

scanf("%d %d", &a, &b);

It reads two integer values from the user.

18.5 Processing Statement

sum = a + b;

The computer adds a and b and stores the result in sum.

18.6 Output Statement

printf("Sum = %d", sum);

It displays the value of sum.

18.7 Return Statement

return 0;

It returns the value 0 from main() to the operating system, conventionally indicating successful termination.

---

19. General Structure of a C Program

A traditional conceptual structure can be represented as:

Documentation / Comments
↓
Preprocessor Directives
↓
Global Declarations
↓
main() Function
↓
{
  Local Declarations
  Executable Statements
}
↓
User-defined Functions

Example

```
/* Program to calculate sum */

#include <stdio.h>

int main()
{
   int a, b, sum;

   a = 10;
   b = 20;

   sum = a + b;

   printf("Sum = %d", sum);

   return 0;
}
```

---

20. Writing a C Program

The first practical step is to write the source code.

A C source file normally uses:

.c

extension.

For example:

hello.c

A simple program:

```
#include <stdio.h>

int main()
{
   printf("Welcome to BIT C Programming");

   return 0;
}
```

21. Compilation

A computer cannot directly execute ordinary C source code.
The C source program must be processed by a **compiler**.

C Source Code
↓
Compiler
↓
Executable Program

For example:

hello.c
↓
Compiler
↓
hello.exe (Windows example)

On Linux, the executable may commonly be named:

hello

---

22. What is a Compiler?

A **compiler** is a software tool that translates source code written in a programming language into a form that can be executed by the computer, while also detecting many syntax and related errors during compilation.

Examples of C compilers include:

- GCC
- Clang
- Microsoft Visual C compiler

---

23. Compilation Process

A simplified C build process is:

Source Code (.c)
↓
Preprocessing
↓
Compilation
↓
Assembly
↓
Linking
↓
Executable

For introductory purposes, students can remember:

Write → Compile → Fix Errors → Run → Test

---

24. Errors in C Programming

Errors are problems in a program that may prevent compilation or cause incorrect behavior.
The major categories introduced at this level are:

24.1 Syntax Error

Occurs when the program violates the syntax rules of C.
Example:

printf("Hello")

The semicolon is missing.
Correct:

printf("Hello");

24.2 Logical Error

The program runs but produces an incorrect result.
Example:
area = length + breadth;
If calculating rectangle area, this is logically incorrect.
Correct:
area = length * breadth;
The compiler may not identify this as an error because the statement is syntactically valid.

24.3 Runtime Error

An error occurs while the program is running.
For example, certain invalid operations or resource-related problems can cause runtime failures.
A classic conceptual example is an invalid division operation:
result = a / b;
when b is zero.

---

25. Debugging

Debugging is the process of finding and correcting errors in a program.
Program
↓
Error
↓
Find Cause
↓
Correct Code
↓
Compile Again
↓
Test Again
Example
Incorrect:
sum = a - b;
Requirement:
Calculate sum
Correction:
sum = a + b;

---

26. Testing

Testing means executing a program with selected input values to check whether it produces the expected output.
Example
Program:
Calculate area = length × breadth
Test case:

Length	Breadth	Expected Area
10	5	50
20	4	80
7	3	21

If the program produces the expected results, the test cases pass.
Important distinction
Debugging = finding and fixing defects.
Testing = checking program behavior/results against expected outcomes.

---

27. Writing and Running C Programs in Windows
There are several ways to develop C programs on Windows.
Option 1: IDE
Students can use an IDE that supports C, such as:

- Code::Blocks
- Dev-C++
- Visual Studio
- Visual Studio Code with a suitable C compiler/toolchain

General process
Create C source file
↓
Write program
↓
Save as program.c
↓
Compile
↓
Correct errors if any
↓
Run
↓
Check output

---

28. Using GCC in Windows
If GCC is installed and available in the terminal, a program can be compiled using:
gcc hello.c -o hello
Then run:
hello
Depending on the Windows shell/environment, the executable may be invoked as:
.\hello.exe

---

29. C Programming in Unix/Linux
Linux and other Unix-like operating systems commonly use GCC or Clang.
Create a file:
hello.c
Compile using GCC:
gcc hello.c -o hello
Run:
./hello
Complete process
$ nano hello.c
↓
Write C program
↓
$ gcc hello.c -o hello
↓

$ ./hello
↓
Output

---

30. Example: Hello World
Source code

```
#include <stdio.h>

int main()
{
   printf("Hello, BIT Students!");
   return 0;
}
```

Linux/Unix compilation

```
gcc hello.c -o hello
```

Execution

```
./hello
```

Output

```
Hello, BIT Students!
```

---

31. Windows vs Unix/Linux

Feature	Windows	Unix/Linux
Source file	.c	.c
Common compiler	GCC, MSVC, Clang	GCC, Clang
Terminal	Command Prompt/PowerShell	Terminal
Example compile	gcc hello.c -o hello	gcc hello.c -o hello
Typical execution	hello.exe / .\hello.exe	./hello
IDEs	Visual Studio, Code::Blocks, VS Code	VS Code, Code::Blocks, terminal editors

The **C language itself remains largely the same**; the compiler, commands, environment and system interfaces may differ.

---

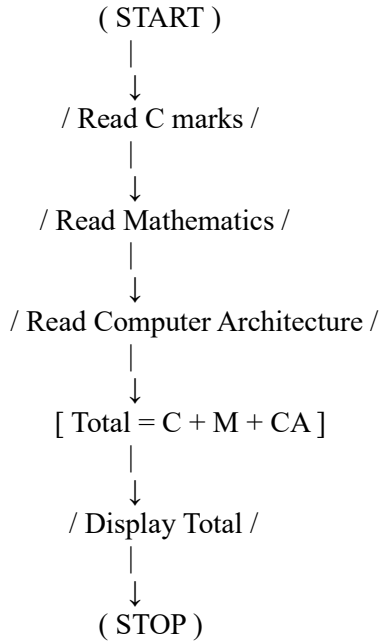
32. Complete Example: Student Marks
Let's apply all Unit 1 concepts.
Problem
Write a C program to calculate the total marks of three subjects.

Problem Analysis
Input:
C Programming marks
Mathematics marks
Computer Architecture marks
Process:
Total = C + Mathematics + Architecture
Output:
Total marks

Algorithm

Step 1: Start
Step 2: Read C marks
Step 3: Read Mathematics marks
Step 4: Read Computer Architecture marks
Step 5: Calculate total
Step 6: Display total
Step 7: Stop

Flowchart



C Program

```
#include <stdio.h>

int main()
{
   int c, math, ca, total;

   printf("Enter C Programming marks: ");
   scanf("%d", &c);

   printf("Enter Mathematics marks: ");
   scanf("%d", &math);

   printf("Enter Computer Architecture marks: ");
   scanf("%d", &ca);

   total = c + math + ca;

   printf("Total Marks = %d", total);

   return 0;
```

```
}
```

Sample Run

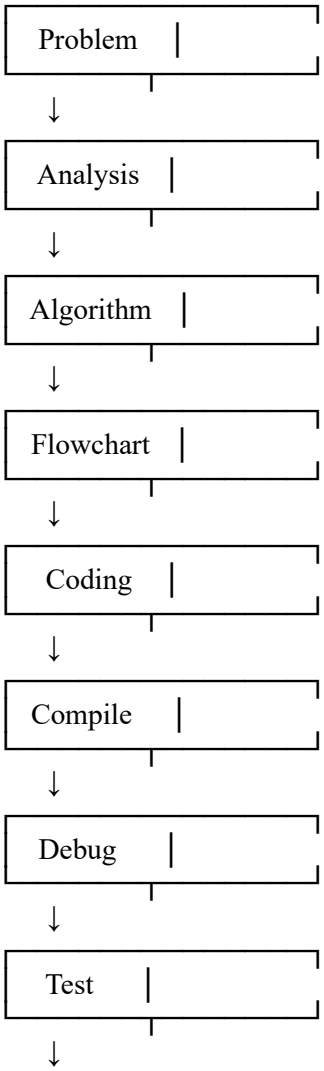
```
Enter C Programming marks: 75
Enter Mathematics marks: 68
Enter Computer Architecture marks: 72

Total Marks = 215
```

---

33. Program Development Cycle

Students should remember the following cycle:



34. Key Terms for Unit 1

Term	Meaning
Program	Set of instructions for a computer
Programming	Process of developing computer programs
C	General-purpose procedural programming language
Algorithm	Step-by-step procedure for solving a problem
Flowchart	Graphical representation of an algorithm
Source code	Program written by the programmer
Compiler	Translates/processes source code to produce executable/object code
Compilation	Process of building a program from source code
Debugging	Finding and correcting program defects
Execution	Running the program
Testing	Checking program behavior using test cases
Syntax error	Violation of language syntax

Logical error	Program runs but produces incorrect result
Runtime error	Problem occurring during execution

---

35. Important Differences

Algorithm vs Program

Algorithm	Program
Step-by-step solution	Actual implementation
Language-independent	Written in a programming language
Usually written before coding	Developed after problem analysis/design
Cannot normally be executed directly	Can be compiled/executed

---

Compiler vs Debugger

Compiler	Debugger
Translates/builds program	Helps investigate program execution
Detects many compile-time errors	Helps locate runtime/logical problems
Produces object/executable output as part of build	Allows controlled execution, inspection and tracing

---

Compilation vs Execution

Compilation:

Source Code → Compiler → Executable

Execution:

Executable → CPU/Operating System → Output

---

36. Laboratory Exercises – Unit 1

Students should perform these exercises practically.

Lab 1: Hello World

Write a C program to display:

Hello, BIT Students!

Welcome to C Programming.

Lab 2: Personal Information

Write a program to display:

Name: Mahesh

Program: BIT

Semester: I

Subject: C Programming

Lab 3: Add Two Numbers

Write a program that:

1. Reads two numbers.
2. Calculates their sum.
3. Displays the result.

Example:

Enter first number: 25

Enter second number: 35

Sum = 60

Lab 4: Rectangle
Write a program to calculate:
- Area
- Perimeter

Formula:
Area = length × breadth

Perimeter = 2 × (length + breadth)

Lab 5: Circle
Write a program to calculate:
- Area
- Circumference

Use:
Area = πr^2

Circumference = $2\pi r$

Lab 6: Student Total and Average
Read marks of five subjects and calculate:
Total
Average

Lab 7: Convert Temperature
Convert Celsius into Fahrenheit.
Formula:
F = (C × 9/5) + 32

Lab 8: Algorithm and Flowchart
For each of the following problems, prepare:
Problem → Input/Process/Output → Algorithm → Flowchart

1. Find the area of a triangle.
2. Calculate simple interest.
3. Calculate the average of three numbers.
4. Convert kilometers to meters.
5. Calculate the total price of five products.

---

37. Short Questions for Examination
Very Short Questions

1. What is C?
2. Who developed C?
3. Where was C developed?
4. What is an algorithm?
5. What is a flowchart?
6. What is a compiler?
7. What is debugging?
8. What is testing?
9. What is the purpose of main()?
10. What is the extension of a C source file?
11. What is the purpose of #include <stdio.h>?
12. What is a syntax error?

13. What is a logical error?
14. What is a runtime error?
15. What is meant by program execution?

---

38. Long/Descriptive Questions

Question 1
Explain the history and major advantages of C programming language.
Question 2
What is an algorithm? Explain its characteristics with a suitable example.
Question 3
What is a flowchart? Explain the commonly used flowchart symbols.
Question 4
Explain the steps involved in developing a C program.
Question 5
Explain the basic structure of a C program with a suitable example.
Question 6
Differentiate between algorithm and flowchart.
Question 7
Explain compilation, debugging, execution and testing of a C program.
Question 8
Explain how a C program can be compiled and executed in Windows and Linux/Unix-like environments.

---

39. Exam-Oriented Summary

For Unit 1, students should remember this sequence:
Problem → Analysis → Algorithm → Flowchart → Coding → Compilation → Debugging → Execution → Testing
C in one view

C
- Developed by Dennis Ritchie
- Bell Laboratories
- Early 1970s
- Procedural / structured programming
- Portable
- Efficient
- Supports low-level operations
- Widely used for system and embedded programming

Problem-solving in one view

INPUT
↓
PROCESS
↓
OUTPUT

C program execution in one view

program.c
↓
Preprocessing
↓
Compilation
↓
Assembly
↓
Linking
↓
Executable
↓
Execution
↓
Output

Key takeaway for programming:

Do not start coding immediately after having a programming problem.
First understand:

1. **What is the problem?**
2. **What are the inputs?**
3. **What processing/formula is required?**
4. **What should be the output?**
5. **How can the solution be represented as an algorithm?**
6. **How can the algorithm be represented as a flowchart?**
7. **How can it be implemented in C?**
8. **How will you compile, debug and test it?**

This problem-solving approach is the foundation for all the later units of **BIT 102 – C Programming**.